School of Future Tech

Case Study Report

On

Math Set Operations Library

by

Tanmay Vijay Sherkar

150096725139

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.

5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Mathematical set operations play a fundamental role in computer science, particularly in areas such as data processing, database systems, and algorithm design. Arrays and collections of data are often manipulated using operations like **Union, Intersection, and Difference** to analyse relationships between datasets.

This case study focuses on designing and implementing a **Mathematical Set Operations Library** using **JavaScript** that performs core set operations on numeric arrays entered by the user. The system accepts user input in the form of comma-separated numbers and dynamically processes the data in a web browser to compute the selected set operation.

The case study demonstrates how **basic programming constructs, array manipulation techniques, and modern JavaScript features** can be applied to build an interactive web-based tool for performing mathematical operations on datasets. It highlights the importance of **input validation, logical comparison, and efficient data handling** in building reliable software applications.

2. Problem Statement / Case Background (Abstract)

Background

In many real-world computing scenarios, developers must process and compare datasets to determine relationships between them. Mathematical set operations provide a structured way to analyse such data. However, performing these operations manually or using static programs is inefficient and prone to error.

A dynamic web-based tool that can perform these operations interactively allows users to quickly analyse relationships between multiple numeric datasets. By implementing these operations using JavaScript, students can gain practical experience with **array manipulation, logical operations, and web-based user interaction**.

Abstract

This case study presents the design and implementation of a **Mathematical Set Operations Library** using JavaScript. The application allows users to input numeric arrays and perform three fundamental set operations: **Union, Intersection, and Difference**.

The system processes user input by converting comma-separated values into numeric arrays, validating the data, and applying set operation logic through reusable JavaScript functions. Modern

ES6 concepts such as **arrow functions, spread operators, and the Set object** are used to improve efficiency and readability.

The application dynamically displays results in the browser using **DOM manipulation techniques**, providing an interactive user experience. The implementation demonstrates how mathematical logic can be integrated with web technologies to create practical data-processing tools.

3. Case Study Design

The **Mathematical Set Operations Library** is designed as a simple web-based system that follows a structured workflow consisting of several stages.

Data Input

The system accepts numeric arrays entered by the user through input fields in the web interface. The values are entered in **comma-separated format**.

Example input:

1, 2, 3, 4

Input Parsing and Validation

The system performs several preprocessing steps to ensure correct data processing:

- Split the input string into individual values using comma delimiters.
- Remove unnecessary spaces using trimming techniques.
- Convert string values into numeric data types.
- Remove or ignore invalid values that cannot be converted into numbers.

Processing and Set Operations

After preprocessing, the arrays are processed using JavaScript functions that implement mathematical set operations:

- **Union** – Combines unique elements from multiple arrays.
- **Intersection** – Identifies common elements between arrays.
- **Difference** – Returns elements present in the first array but not in the second.

The system ensures that duplicate elements are removed where required to maintain proper **set properties**.

Result Display

The application dynamically displays the results using **DOM manipulation**. The selected operation and the resulting array are shown instantly without reloading the page.

4. Methods & Algorithms Technology Applied in the Case Study

Key Methods and Algorithms

Input Processing Techniques

- Splitting comma-separated values
- Trimming whitespace from inputs
- Converting strings into numeric values
- Filtering invalid inputs

Array Processing Methods

The system uses JavaScript array methods to perform comparisons and filtering.

Examples include:

- Filtering elements
- Checking element presence in arrays
- Removing duplicates

Mathematical Set Logic

The project implements three fundamental set operations:

- **Union** – Combine elements from both arrays while maintaining uniqueness.
- **Intersection** – Find elements that appear in both arrays.
- **Difference** – Identify elements that exist in one array but not in another.

Technology Stack Used

Programming Language

- JavaScript

Web Technologies

- HTML5 – For webpage structure and user interface
- CSS – For styling and layout of the application

JavaScript Concepts Used

- Arrays for data storage and processing
- Set object to ensure uniqueness of elements
- Arrow functions for concise function syntax
- Spread operator for combining arrays
- Template literals for displaying output

Development Environment

The project can be developed and tested using:

- Visual Studio Code
- Any modern web browser (Chrome, Edge, Firefox)

5. Problem Statement / Case Study Implementation Details and Snapshots

The implementation of the project is organized into three main files.

index.html

Defines the structure of the webpage including:

- Input fields for numeric arrays
- Dropdown menu for selecting set operations
- Button to execute the operation
- Area to display results

style.css

Responsible for the visual design of the interface including:

- Layout and spacing
- Font styles and colors
- Button and input styling

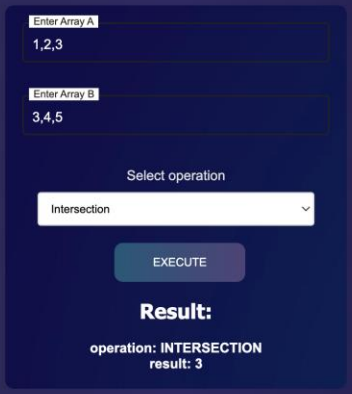
script.js

Contains the core logic of the application including:

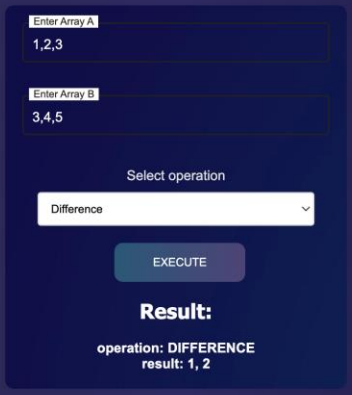
- Functions for parsing user input
- Implementation of union, intersection, and difference operations
- Event listeners for user interaction
- DOM manipulation for displaying results

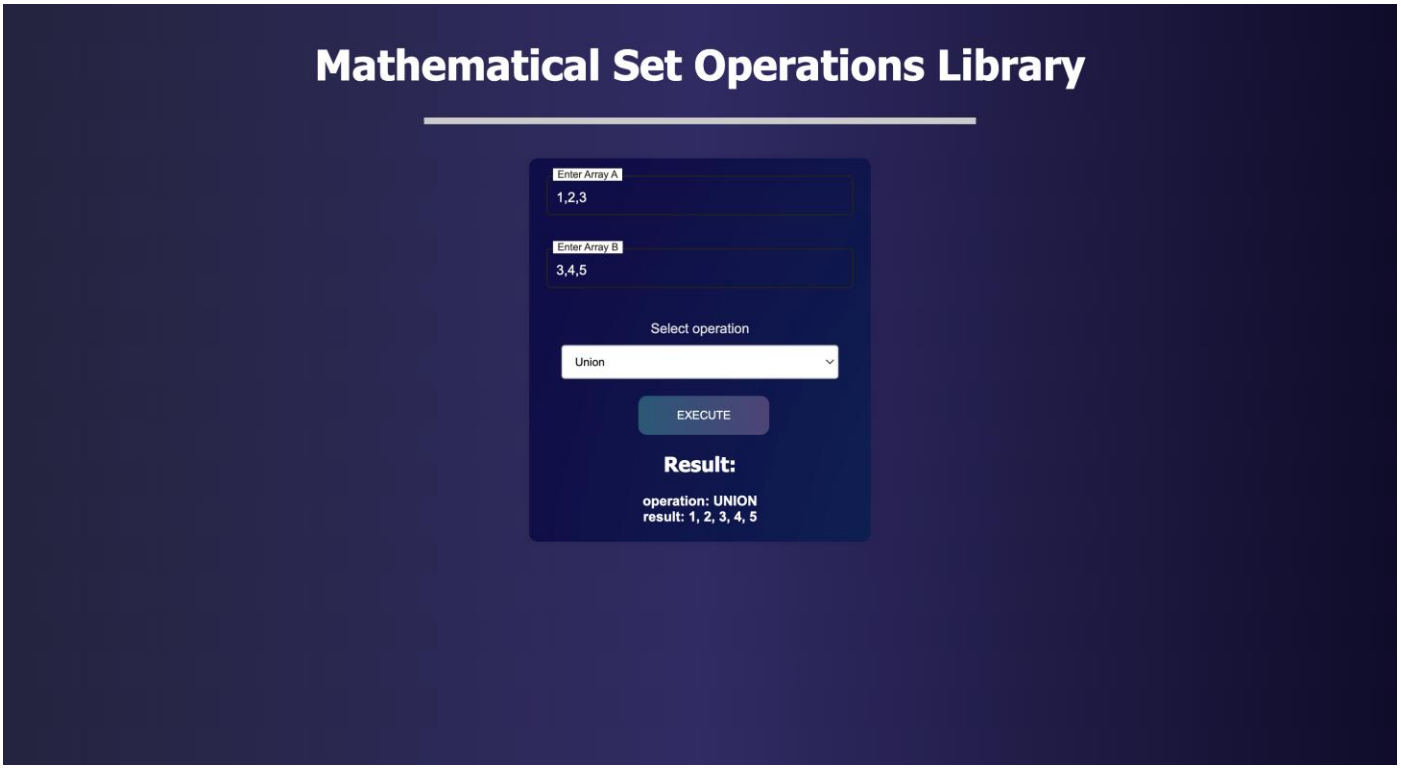
Snapshots of UI

Mathematical Set Operations Library



Mathematical Set Operations Library





6. Deployment and Live Application

To make the application easily accessible, the project was deployed on a cloud hosting platform. Deployment allows users to access the application directly through a web browser without installing any software locally.

The application is hosted online and can be accessed through the following link:

Deployment Link:
https://full-stack-dev-woad.vercel.app

This deployment enables users to test the functionality of the **Mathematical Set Operations Library** in a real-time web environment. The hosted version provides the same features as the local version, including input of numeric arrays, selection of set operations, and dynamic display of results.

7. Problem Statement / Case Study Results and Conclusion

Findings

• JavaScript array methods provide efficient tools for performing mathematical operations on datasets.
• Using the **Set object** simplifies duplicate removal in union operations.

- Proper input validation ensures accurate computation and prevents runtime errors.
- Dynamic DOM manipulation allows results to be displayed instantly, improving user interaction.

Conclusion

This case study successfully demonstrates how fundamental mathematical set operations can be implemented using **JavaScript and modern web technologies**. The developed system accepts numeric arrays as input, processes them using logical set operations, and dynamically displays the results in a user-friendly interface.

The project provides practical experience in **array processing, input validation, algorithm implementation, and interactive web development**. It also highlights how mathematical concepts can be translated into efficient computational solutions.

Overall, the Mathematical Set Operations Library serves as a simple yet effective tool for understanding **set theory concepts and their applications in programming and data processing**.

8. References

- JavaScript Documentation – Mozilla Developer Network (MDN)
- HTML5 and CSS documentation – W3Schools
- ECMAScript (ES6) feature documentation
- JavaScript Array Methods Reference – MDN Web Docs
- Educational materials on mathematical set theory and data structures